Using Claude AI

Prompt 1: Make a Python script that will produce all of the deliverables in the file. The dataset is in this file path: "C:\Users\Camille\Downloads\Data 01.xlsx".

I'll build this as a complete, runnable Python script that produces the HTML dashboard per the spec. Since I don't have access to your actual `Data 01.xlsx` (it's on your local machine, not uploaded here), I'll write the script with robust auto-detection of your column names, then test it against a synthetic reservoir dataset in my sandbox to make sure it runs cleanly before handing it to you.Good, all needed libraries are available. Let me build a synthetic test dataset that mimics the described reservoir stage log (96 readings, 15-min sampling, one filling event, cm rounding) to test the script against.Now let me write the main lab script.Now let's test this against the synthetic data to make sure it runs cleanly end-to-end.It runs cleanly and recovers the true parameters accurately (c≈12.29 vs true 12.30, a≈3.86 vs 3.85, k≈0.75 vs 0.75, t0≈11.49 vs 11.5). Let me verify the HTML output renders correctly.Let me visually confirm the dashboard renders correctly by converting it to an image.All 5 plots embedded correctly, tabs work, no leftover template placeholders. Now let's finalize and present the real script (with your actual file path) as the deliverable.I tested it against a synthetic reservoir dataset (same shape as your real one: 96 readings, 15-min sampling) to confirm it runs end-to-end — the sample HTML is the actual output from that test run, so you can preview the dashboard layout before pointing it at your real file.

Already set: `DATA_PATH` points to `C:\Users\Camille\Downloads\Data 01.xlsx`, and it auto-detects your timestamp and water-level columns by name/dtype (if detection guesses wrong, there's a fallback spot to hardcode `TIME_COL`/`LEVEL_COL` right in `load_stage_log()`).

What it does, following the sheet's order:

- Converts timestamps to hours-from-first-reading, computes forward/backward/central first and second finite differences, reports max dh/dt and when it happens
- Fits all four candidate models (logistic, Gompertz, sum-of-two-logistics, cubic) with `curve_fit(..., method="lm")`, using data-derived `p0` (never default ones), and prints a console comparison table so you can pick and defend a model — `CHOSEN_MODEL` at the top controls which one drives the dashboard
- Computes SSE, SST, R^2, s, per-parameter SE/t/p exactly per the formulas in the sheet
- Integrates with `scipy.integrate.quad` (reports the abs-error estimate) and cross-checks with `np.trapezoid`
- Builds one self-contained HTML file — charts are matplotlib PNGs embedded as base64, so there's no CDN dependency and it opens by double-click, offline, no server

One thing left to you, on purpose: the `NOTES` dict near the top has `TODO` placeholders for the written interpretation (residual reading, second-derivative meaning, model justification, area meaning). Those are explicitly the "say in one sentence / defend your choice" parts of the assignment — the professor wants *your* reading of the evidence, so I left them as

clearly-marked fields for you to fill in rather than guessing at prose on your behalf. Everything numeric is fully automated.

Rename the two files to `lab02_<YourSurname>.py`/`.html` before committing — the `results.pdf` and `prompts.pdf` deliverables are separate write-ups you'll still need to assemble yourself.

Prompt 2: I need the script so I can run it on my own.

You've already got it — `lab02_yourname.py` in the file card above is the full, standalone script. Just:

1. Download it and open it in a text editor.
2. Confirm `DATA_PATH` at the top points to your file: `C:\Users\Camille\Downloads\Data 01.xlsx` (already set).
3. Make sure the libraries are installed:

pip install numpy pandas scipy matplotlib openpyxl

4. Run it:

python lab02_yourname.py

It'll print a model-comparison table and summary stats to the console, and write `lab02_yourname.html` in the same folder — that's your dashboard.

Two things worth checking on your first run:

- The console line `[load] time column: '...' level column: '...'` — make sure it picked the right two columns from your spreadsheet. If it guessed wrong, open the script and hardcode `TIME_COL` / `LEVEL_COL` inside `load_stage_log()`.
- Before you submit, fill in the `TODO` placeholders in the `NOTES` dict near the top with your own reading of the residuals, the second derivative, your model choice, and the area — those are meant to be your analysis, not auto-generated.

If it errors out on your real data (e.g. a model fails to converge, or column detection fails), paste me the console output and I'll help you fix it.

Prompt 3: PS C:\Users\Camille\Downloads\python-practice> & "C:\Program Files\Python314\python.exe" c:/Users/Camille/Downloads/python-practice/lab02_dacillo.py Traceback (most recent call last): File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 624, in <module> build_dashboard(DATA_PATH) ~~~~~~~~~~~~~~~^^^^^^^^^^ File

Using Claude AI

"c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 531, in build_dashboard t, h, timestamps = load_stage_log(path) ~~~~~~~~~~~~~~^^^^^^ File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 121, in load_stage_log raise ValueError(...<2 lines>...) ValueError: Could not auto-detect columns. Found columns: ['RESERVOIR STAGE RECORD, 15-MINUTE LOGGER', 'Unnamed: 1', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']. Set TIME_COL and LEVEL_COL manually inside load_stage_log().

Ah, this is a common Excel layout issue — your file has a title row (`RESERVOIR STAGE RECORD, 15-MINUTE LOGGER`) merged across the top, so pandas grabbed that as the header instead of your real column names. Let me fix the loader to auto-detect where the actual table starts, regardless of title rows.Now let me rewrite the loader with content-based auto-detection that scans for where the actual data table begins, regardless of title/blank rows above it.Now let's test against both the original simple layout and the new title-row layout mimicking Camille's actual file.Good, still works. Now the harder test — the title-row layout matching Camille's actual error:It correctly skipped the merged title row and blank spacer row, found the real header at row 2, correctly picked `Timestamp` and `Stage (m)` out of 5 columns (ignoring `Reading No.`, `Logger ID`, `Notes`), and matched the exact same fit results as the clean-layout test. Let me also test the edge case with no separate header row at all, and check the HTML outputs are valid.Both work cleanly. Let's ship the fixed version.Fixed — your file has a merged title row (`RESERVOIR STAGE RECORD, 15-MINUTE LOGGER`) sitting above the real table, which was tricking pandas into treating that banner text as the column header. The loader now:

- Scans the first 20 rows for wherever a real timestamp + a real number first show up together, and treats everything above that as banner/blank rows to skip
- Picks up the actual header labels right above the data if there are any
- Detects the timestamp and level columns by what the values actually parse as, not just by column-name guessing — so it'll correctly ignore extra columns like a reading-number index or notes column

I tested it against a mock file shaped like yours (title row → blank row → header row → data, with extra columns thrown in) and it correctly found `data starts at file row: 3` and picked the right two columns out of five.

Re-download `lab02_yourname.py` from the card above (same filename, updated content) and rerun it — same command as before:

python "c:/Users/Camille/Downloads/python-practice/lab02_dacillo.py"

(you'll want to rename the downloaded file back to `lab02_dacillo.py` to overwrite the old one, or just replace its contents). If it still trips on your real file, paste me the new console output — it now also prints which row it picked as the header and where data starts, which will make it easy to spot if it guessed wrong.

Using Claude AI

Prompt 4: PS C:\Users\Camille\Downloads\python-practice> & "C:\Program Files\Python314\python.exe" c:/Users/Camille/Downloads/python-practice/lab02_dacillo.py Traceback (most recent call last): File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 624, in <module> build_dashboard(DATA_PATH) ~~~~~~~~~~~~~~~~^^^^^^^^^^ File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 531, in build_dashboard t, h, timestamps = load_stage_log(path) ~~~~~~~~~~~~~~^^^^^^ File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 121, in load_stage_log raise ValueError(...<2 lines>...) ValueError: Could not auto-detect columns. Found columns: ['RESERVOIR STAGE RECORD, 15-MINUTE LOGGER', 'Unnamed: 1', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4']. Set TIME_COL and LEVEL_COL manually inside load_stage_log(). PS C:\Users\Camille\Downloads\python-practice>

That error message and line numbers are identical to before — which tells me the file on your machine is still the **old** version. The new script I sent doesn't even raise that exact error anymore (it now says *"Could not find a row that looks like actual data..."* with a preview of your file), so this confirms `lab02_dacillo.py` on disk wasn't actually replaced.

This usually happens when the download saves as `lab02_yourname.py` or `lab02_yourname (1).py` in your Downloads folder instead of overwriting `lab02_dacillo.py` in `python-practice`.

To fix it:

1. Go to your Downloads folder and find the file you just downloaded from this chat (it'll be named `lab02_yourname.py`, possibly with `(1)` or similar appended).
2. Open it and check — near the top, in `load_stage_log()`, it should have a block starting with `def _cell_is_datetime(val):`. If that function isn't there, you have the old file.
3. Copy that file into `C:\Users\Camille\Downloads\python-practice\`, replacing `lab02_dacillo.py` entirely (delete the old one first, or overwrite it), then rename it to `lab02_dacillo.py`.
4. Re-run:

python c:/Users/Camille/Downloads/python-practice/lab02_dacillo.py

Quick sanity check before you rerun — open the file in a text editor and search for the text `"Could not find a row that looks like actual data"`. If that phrase is in there, you have the right version. If it's not, you're still looking at the old copy.

Prompt 5: Name the standalone html as lab02_dacillo, the html should open even when offline. Make this color #E89EB8 the main theme then use other colors that will

complement it. The script should also produce the lab02_dacillo_results.pdf containing one page - fitted parameters with standard errors, t and p, SSE, R2, s, the area with its units, and your reading of the residuals.

Now let's implement the color theme, PDF generation, and filename changes.Now update the config section for filenames:Now let's update the matplotlib plot functions to use the theme colors:Now update the HTML template's CSS to use the theme:Now let's add the PDF generation function:Now let's wire the PDF call into `build_dashboard`:Now let's rename the file itself to match the deliverable naming, and run the full test:Both files generated successfully. Let's check the PDF is truly one page and verify its content, and check the HTML looks right.Exactly one page, all required content present. Let's render it as an image to visually check layout and confirm the HTML dashboard looks good too.That looks clean, one page, and matches the pink/teal theme. Now let's check the HTML dashboard visually.This looks great — pink header with the deeper rose gradient, teal-green complement for the second derivative and note boxes. Let's check the other two tabs.Excellent, clean and cohesive. Let's check the third tab too.All three tabs look cohesive and on-theme. The dashboard uses only inline base64-embedded images and no external resources, so it's fully offline-capable — let me double check there's zero network dependency in the HTML.No external URLs at all — just the one inline `<script>` for tab-switching. Everything's self-contained. Let's finalize and ship the deliverables.Done. Three changes, all tested end-to-end:

1. Filenames — the script now writes `lab02_dacillo.html` and `lab02_dacillo_results.pdf` directly (no renaming needed on your end).

2. Color theme — built around `#E89EB8`:

- Deep rose (`#C9749B` / `#7A3B57`) for the header, headings, and primary chart lines
- Teal-green (`#4FA88A` / `#357A63`) as the complement — used for the second-derivative plot, the fitted curve, and the note boxes so they stand apart from the pink data series
- Soft pink page background, white cards, warm plum text color

Applied consistently across the HTML dashboard *and* the matplotlib plots *and* the PDF tables, so all three deliverables visually match.

3. `lab02_dacillo_results.pdf` — one page, confirmed via `pypdf`, containing exactly what you asked for: fitted parameters with SE/t/p, SSE, R², s, the area with units (plus the trapezoid cross-check), and a residuals section. That residuals section pulls from the same `NOTES` dict as the dashboard — right now it shows the `TODO` placeholders, so once you fill in your actual reading of the residuals near the top of the script, both the HTML and this PDF update together automatically.

The HTML stays fully offline — everything (all 5 charts) is embedded as inline base64 PNGs, only one local `<script>` for tab-switching, zero external URLs.

Using Claude AI

One thing to flag: I built and tested this against a synthetic file shaped like yours (title row above the header), not your actual `Data 01.xlsx`, since I don't have access to it. Run it and send me the console output if anything doesn't line up.

Prompt 6: PS C:\Users\Camille\Downloads\python-practice> & "C:\Program Files\Python314\python.exe" c:/Users/Camille/Downloads/python-practice/lab02_dacillo.py Traceback (most recent call last): File "c:\Users\Camille\Downloads\python-practice\lab02_dacillo.py", line 40, in <module> from reportlab.lib.pagesizes import letter ModuleNotFoundError: No module named 'reportlab'

Easy fix — `reportlab` (used to build the PDF) just isn't installed on your machine yet. Run this in PowerShell:

pip install reportlab

If that doesn't work (e.g. multiple Python versions installed), target the exact interpreter you're using:

"C:\Program Files\Python314\python.exe" -m pip install reportlab

Then rerun the script the same way as before. If you get a similar `ModuleNotFoundError` for anything else (`scipy`, `matplotlib`, `pandas`, `openpyxl`), install those the same way — the script needs all of:

"C:\Program Files\Python314\python.exe" -m pip install numpy pandas scipy matplotlib openpyxl reportlab